

```

21     assertNotNull(result);
22     assertEquals(expected: 0, result.sum());
23     assertEquals(new Triple(x: -1, y: 0, z: 1), result);
24 }
25
26 /**
27  * Test case for getTriple method when no valid triple exists.

```

Run: ThreeSumQuadrithmicTest.testGetTriple_ValidTriple

✓ ThreeSumQuadrithmic 2 ms | ✓ 1 test passed | 1 test total, 2 ms

✓ testGetTriple_ValidTriple 2 ms

/Users/liang.zim/Library/Java/JavaVirtualMachines/ms-21.0.9/Contents/Home/bin/java ...

Process finished with exit code 0

N	Cubic (ms)	Quadrithmic (ms)	Quadratic (ms)
250	9	4	3
500	23	6	3
1000	148	12	7
2000	1077	54	37
4000	8483	209	201

```

15 //
16
17 @Test
18 public void testGetTriplesWithValidTriplets() {
19     int[] inputArray = {-4, -1, -1, 0, 1, 2};
20     ThreeSum threeSum = new ThreeSumQuadratic(inputArray);
21     Triple[] result = threeSum.getTriplets();

```

Run: ThreeSumQuadraticTest

✓ ThreeSumQuadratic 81 ms | ✓ 11 tests passed | 11 tests total, 81 ms

✓ testGetTriplesWithD 3 ms

✓ testGetTriplesWithN 0 ms

✓ testGetTriplesWithP 0 ms

✓ testGetTriplesWithE 1 ms

✓ testGetTriplesWithT 0 ms

/Users/liang.zim/Library/Java/JavaVirtualMachines/ms-21.0.9/Contents/Home/bin/java ...

[Triple{x=-1, y=-1, z=2}, Triple{x=-1, y=0, z=1}]

Process finished with exit code 0

The Quadratic method reduces the time complexity from $O(n^3)$ to $O(n^2)$. $a[j]$ is already fixed, and i and k move from both sides toward the middle. If $\text{sum} < 0$, the sum is too small, move the left pointer right; if $\text{sum} > 0$, the sum is too large, move the right pointer left; if $\text{sum} == 0$, the target is found. This way, the third loop is not needed. The Quadrithmic method has a time complexity of $O(n^2 \log n)$, requires $\log n$ comparison operations, and is also slower than Quadratic.